

AI ASSISTANT CODING ASSIGNMENT-17.2

Name: A.Sumanth

Enrollment no: 2303A52191

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task Description – 1(Student Academic Data Cleaning)

- Task: Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd  
  
data = pd.read_csv("students.csv")  
  
print(data.head())
```

Expected Output:

- A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.

The top screenshot shows the Jupyter Notebook code for creating a dummy dataset, loading it, and cleaning it. The code includes the following steps:

```
import pandas as pd
import os
if not os.path.exists("students.csv"):
    print("students.csv not found. Creating a dummy file for demonstration.")
    dummy_data = {
        'student_id': [1, 2, 3, 4, 5, 6],
        'marks': [85, 78, None, 92, 78, 65],
        'attendance': [98, 85, 78, None, 95, 88],
        'department': ['cse', 'ece', 'CSE', 'EEE', 'CS', None],
        'gender': ['male', 'female', 'male', 'female', 'male', 'female']
    }
    pd.DataFrame(dummy_data).to_csv("students.csv", index=False)
# Load dataset
data = pd.read_csv("students.csv")
display(students_df)
print("Original Data:\n", data)
data['marks'] = data['marks'].fillna(data['marks'].mean())
data['attendance'] = data['attendance'].fillna(data['attendance'].median())
data['department'] = data['department'].fillna(data['department'].mode()[0])
data = data.drop_duplicates(subset='student_id')
data['department'] = data['department'].str.lower().str.strip()
data['department'] = data['department'].replace({
    'cse': 'computer science',
    'cs': 'computer science',
    'ece': 'electronics',
    'eee': 'electrical'
})
data['gender'] = data['gender'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['department'])
print("\nCleaned Data:\n", data)
data.to_csv("cleaned_students.csv", index=False)
```

The bottom screenshot shows the output of the code, including the original and cleaned data tables.

Original Data:

student_id	marks	attendance	department	gender	
0	1	85.0	98.0	cse	male
1	2	78.0	85.0	ece	female
2	3	NaN	78.0	CSE	male
3	4	92.0	NaN	EEE	female
4	5	78.0	95.0	CS	male
5	6	65.0	88.0	NaN	female

Cleaned Data:

student_id	marks	attendance	gender	department_computer science \
0	1	85.0	0	True
1	2	78.0	1	False
2	3	78.0	0	True
3	4	92.0	1	False
4	5	78.0	0	True
5	6	65.0	1	True

department_electrical department_electronics

	department_electrical	department_electronics
0	False	False
1	False	True
2	False	False
3	True	False
4	False	False
5	False	False

Task Description – 2 (Financial Transaction Data Preprocessing)

- **Task:** Preprocess financial transaction data using AI-generated Python code.

Instructions:

- Convert transaction date columns to datetime format.
- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Sample Code:

```
import pandas as pd
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
transactions = pd.read_csv("transactions.csv")
```

```
print(transactions.info())
```

Expected Output:

- A transformed dataset with normalized values and newly generated time based features.

The image displays three sequential screenshots of a Jupyter Notebook interface, illustrating the steps to transform a dataset.

Top Screenshot: Shows the initial code cells. The first cell imports `pandas` and `MinMaxScaler`. The second cell creates a dataset with 12 transactions, each having a unique ID, a date, an amount, and a type (credit or debit). The third cell converts the date to a datetime format and extracts year and month features. The fourth cell handles extreme values by calculating the IQR and clipping the 'amount' column.

Middle Screenshot: Shows the continuation of the transformation. The fifth cell normalizes the 'amount' column using `MinMaxScaler`. The sixth cell saves the transformed data to a CSV file named 'processed_transactions.csv' and downloads it. Below the code, the 'Original Data' is displayed as a table:

	transaction_id	transaction_date	amount	type
0	1	2023-01-15	5000	credit
1	2	2023-02-20	15000	debit
2	3	2023-03-05	250000	credit
3	4	2023-04-10	7000	debit
4	5	2023-05-25	12000	credit
5	6	2023-06-30	999999	debit
6	7	2023-07-10	3000	credit
7	8	2023-08-12	4500	debit
8	9	2023-09-01	200000	credit
9	10	2023-10-22	8000	debit
10	11	2023-11-05	6000	credit
11	12	2023-12-15	17500	debit

Below this, the 'Processed Data' is shown as a table with additional 'year' and 'month' columns:

	transaction_id	transaction_date	amount	type	year	month
0	1	2023-01-15	5000.0	credit	2023	1
1	2	2023-02-20	15000.0	debit	2023	2
2	3	2023-03-05	149187.5	credit	2023	3
3	4	2023-04-10	7000.0	debit	2023	4
4	5	2023-05-25	12000.0	credit	2023	5
5	6	2023-06-30	149187.5	debit	2023	6
6	7	2023-07-10	3000.0	credit	2023	7
7	8	2023-08-12	4500.0	debit	2023	8
8	9	2023-09-01	149187.5	credit	2023	9
9	10	2023-10-22	8000.0	debit	2023	10
10	11	2023-11-05	6000.0	credit	2023	11
11	12	2023-12-15	17500.0	debit	2023	12

Bottom Screenshot: Shows the final output, a table of the normalized 'amount' values:

	amount_normalized
0	0.013681
1	0.082086
2	1.000000
3	0.027362
4	0.061565
5	1.000000
6	0.000000
7	0.010261
8	1.000000
9	0.034203
10	0.020622
11	0.099188

Task 3 – (Environmental Sensor Data Preparation)

- Task: Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Sample Code:

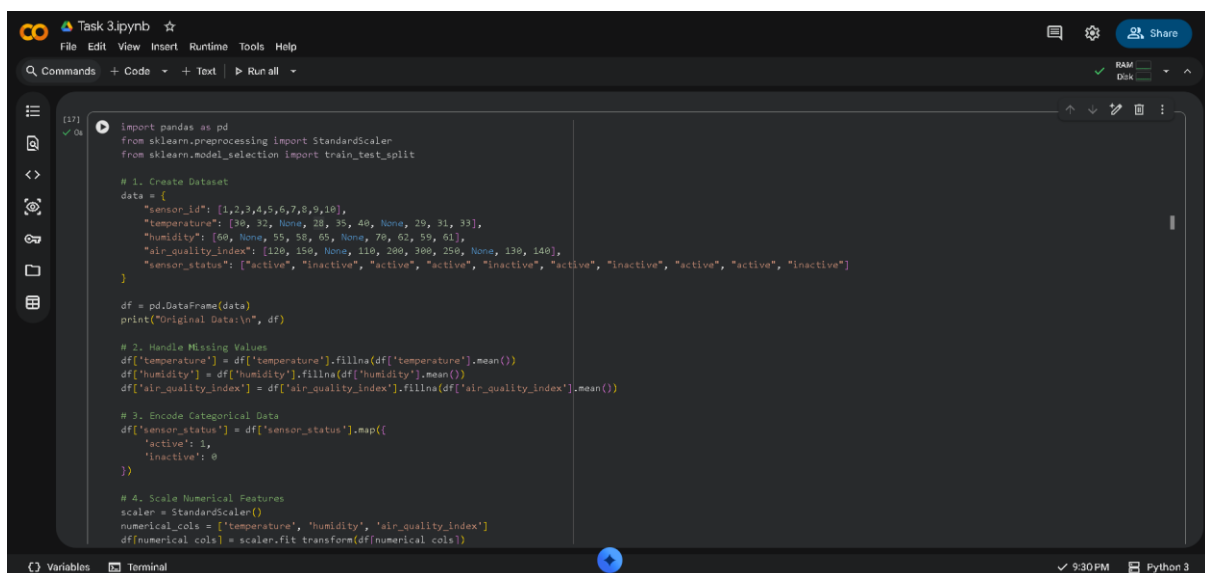
```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("sensor_data.csv")
```

Expected Output:

- A fully preprocessed dataset ready for predictive modeling tasks.



The screenshot shows a Jupyter Notebook titled "Task 3.ipynb" with a dark theme. The code is written in a Python cell and includes the following steps:

```
(12) import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Create Dataset
data = {
    "sensor_id": [1,2,3,4,5,6,7,8,9,10],
    "temperature": [30, 32, None, 25, 35, 40, None, 29, 31, 33],
    "humidity": [60, None, 55, 58, 65, None, 70, 62, 59, 61],
    "air_quality_index": [120, 150, None, 110, 200, 300, 250, None, 130, 140],
    "sensor_status": ["active", "inactive", "active", "active", "inactive", "active", "inactive", "active", "active", "inactive"]
}

df = pd.DataFrame(data)
print("Original Data:\n", df)

# 2. Handle Missing Values
df["temperature"] = df["temperature"].fillna(df["temperature"].mean())
df["humidity"] = df["humidity"].fillna(df["humidity"].mean())
df["air_quality_index"] = df["air_quality_index"].fillna(df["air_quality_index"].mean())

# 3. Encode Categorical Data
df["sensor_status"] = df["sensor_status"].map({
    'active': 1,
    'inactive': 0
})

# 4. Scale Numerical Features
scaler = StandardScaler()
numerical_cols = ["temperature", "humidity", "air_quality_index"]
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

The interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. A left sidebar contains icons for file management and execution. The bottom status bar shows "Variables", "Terminal", and the current time "9:30 PM" along with the Python version "Python 3".

```
# 5. Split Dataset
X = df.drop(columns=['sensor_id']) # Features
y = df['sensor_status'] # target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Output Results
print("\nPreprocessed Data:\n", df)
print("\nTraining Set Shape:", X_train.shape)
print("\nTesting Set Shape:", X_test.shape)
```

Original Data:

	sensor_id	temperature	humidity	air_quality_index	sensor_status
0	1	30.0	60.0	120.0	active
1	2	32.0	NaN	150.0	inactive
2	3	NaN	55.0	NaN	active
3	4	28.0	58.0	110.0	active
4	5	35.0	65.0	200.0	inactive
5	6	40.0	NaN	300.0	active
6	7	NaN	70.0	250.0	inactive
7	8	29.0	62.0	NaN	active
8	9	31.0	59.0	130.0	active
9	10	33.0	61.0	140.0	inactive

Preprocessed Data:

	sensor_id	temperature	humidity	air_quality_index	sensor_status
0	1	-0.699379	-0.325472	-0.957427	1
1	2	-0.077709	0.000000	-0.435194	0
2	3	0.000000	-1.627361	0.000000	1
3	4	-1.021040	-0.040328	-1.131505	1
4	5	0.054790	0.970417	0.435194	0
5	6	2.400971	0.000000	2.175971	1
6	7	0.000000	2.278306	1.305582	0
7	8	-1.010214	0.195283	0.000000	1
8	9	-0.308544	-0.505080	-0.703349	1
9	10	0.233120	-0.000004	-0.609272	0

Training Set Shape: (8, 4)
Testing Set Shape: (2, 4)

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

• Scenario:

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```
import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output:

- A cleaned and normalized traffic dataset with an accompanying

data-quality improvement summary.

The image displays three sequential screenshots of a Jupyter Notebook titled "Task 4.ipynb", illustrating the steps for data quality improvement on a traffic dataset.

First Screenshot: The notebook starts with importing pandas and MinMaxScaler. A sample dataset is created with columns: sensor_id, road_name, location, traffic_density, speed, and vehicle_count. The initial data is printed, showing some missing values (NaN) and duplicates. The first two steps are: 1. Create Sample Dataset and 2. Data Quality BEFORE Cleaning (showing missing values and duplicate rows).

Second Screenshot: This section continues with data cleaning steps: 3. Standardize Text Fields (lowercase and strip whitespace), 4. Handle Missing Values (fillna with median), 5. Remove Duplicates (drop_duplicates), and 6. Normalize Numerical Features (MinMaxScaler). The data is printed after each step to show the progress.

Third Screenshot: The final steps are: 7. Data Quality AFTER Cleaning (showing no missing values or duplicates) and 8. Final Output (printing the cleaned data). The notebook concludes by saving the cleaned dataset to a CSV file.

Original Data:

sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	M.G. Road	Down town	50.0	45
1	2	mg road	down town	NAN	50
2	3	M.G. Road	DOWN TOWN	55.0	48
3	4	Highway 1	Uptown	80.0	60
4	5	highway-1	uptown	NAN	62
5	6	Main St	Midtown	40.0	35
6	7	main street	mid town	42.0	38
7	8	MAIN ST	MIDTOWN	NAN	36
8	9	Ring Rd	Suburb	60.0	55
9	10	ring road	suburb	65.0	58
10	2	mg road	down town	50.0	50

BEFORE CLEANING SUMMARY

Missing Values:

sensor_id	0
road_name	0
location	0
traffic_density	3
speed	0
vehicle_count	0

dtype: Int64

Duplicate Rows: 0

The screenshot shows a Jupyter Notebook titled 'Task 4.ipynb'. The main content area displays the output of a data cleaning process. It includes a section titled 'AFTER CLEANING SUMMARY' with a green checkmark icon. Below this, it lists 'Missing Values' for various columns: sensor_id, road_name, location, traffic_density, speed, and vehicle_count, all showing 0. It also indicates 'Duplicate Rows: 0'. A second section, 'Cleaned Data:', shows a table with 11 rows and 6 columns: sensor_id, road_name, location, traffic_density, speed, and vehicle_count. The data is as follows:

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	mg road	downtown	50.0	0.370970	0.185185
1	2	mg road	downtown	52.5	0.555556	0.259259
2	3	mg road	downtown	55.0	0.481481	0.222222
3	4	highway 1	uptown	80.0	0.925926	0.925926
4	5	highway 1	uptown	52.5	1.000000	1.000000
5	6	main st	midtown	40.0	0.000000	0.000000
6	7	main st	midtown	42.0	0.111111	0.037037
7	8	main st	midtown	52.5	0.037037	0.018519
8	9	ring rd	suburb	60.0	0.740741	0.555556
9	10	ring rd	suburb	65.0	0.851852	0.629630
10	2	mg road	downtown	50.0	0.555556	0.259259

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization. Generate basic text features such as word count and sentiment score.

Sample Code:

```
import pandas as pd

import re

posts = pd.read_csv("social_media_posts.csv")

print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.

Task 5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

119

✓ 5s

```
!pip install nltk textblob
```

Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.19.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)

221

✓ 0s

```
import pandas as pd
import re
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from nltk.stem import PorterStemmer
from textblob import TextBlob

# 1. Create Sample Dataset
data = {
    "post_id": [1,2,3,4,5,6,7,8,2],
    "text": [
        "I love this product! 🍌 http://example.com",
        "Worst service ever!!! 🍌",
        None,
        "Amazing experience, will buy again!!!",
        "Check this out: https://text.com #awesome",
        "Not good, very bad service",
        "OK product, nothing special",
        "I LOVE THIS PRODUCT! 🍌 http://example.com",
        "Worst service ever!!! 🍌"
    ]
}
```

Variables Terminal

9:46 PM Python 3

Task 5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

221

✓ 0s

```
posts = pd.DataFrame(data)
print("🍌 Original Data:\n", posts)

# 2. Remove Duplicates & Nulls
posts = posts.drop_duplicates(subset='text')
posts = posts.dropna(subset='text')

# 3. Clean Text
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http[s]*", "", text) # remove URLs
    text = re.sub(r"[a-z]!", "", text) # remove special chars
    text = re.sub(r"^\s+|\s+$", "", text).strip() # remove extra spaces
    return text

posts['clean_text'] = posts['text'].apply(clean_text)

# 4. Tokenization
posts['tokens'] = posts['clean_text'].apply(lambda x: x.split())

# 5. Remove Stopwords
posts['tokens'] = posts['tokens'].apply(
    lambda words: [w for w in words if w not in ENGLISH_STOP_WORDS]
)

# 6. Stemming
stemmer = PorterStemmer()
posts['tokens'] = posts['tokens'].apply(
    lambda words: [stemmer.stem(w) for w in words]
)

# Join tokens back
posts['processed_text'] = posts['tokens'].apply(lambda x: " ".join(x))
```

Variables Terminal

9:46 PM Python 3

Task 5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

221

✓ 0s

```
# Join tokens back
posts['processed_text'] = posts['tokens'].apply(lambda x: " ".join(x))

# 7. Feature Extraction

# Word count
posts['word_count'] = posts['processed_text'].apply(lambda x: len(x.split()))

# Sentiment score
posts['sentiment_score'] = posts['processed_text'].apply(
    lambda x: TextBlob(x).sentiment.polarity
)

# 8. Final Output
print("\n🍌 Cleaned & Processed Data:\n", posts)

# Save file
posts.to_csv("cleaned_social_media_posts.csv", index=False)
```

...

🍌 Original Data:

post_id	text
0	1 I love this product! 🍌 http://example.com
1	2 Worst service ever!!! 🍌
2	3 None
3	4 Amazing experience, will buy again!!!
4	5 Check this out: https://text.com #awesome
5	6 Not good, very bad service
6	7 OK product, nothing special
7	8 I LOVE THIS PRODUCT! 🍌 http://example.com
8	2 Worst service ever!!! 🍌

🍌 Cleaned & Processed Data:

post_id	text
0	1 I love this product! 🍌 http://example.com

Variables Terminal

9:46 PM Python 3

Task 5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

RAM 100% Disk 100%

Share

Original Data:

post_id	text
1	I love this product!  http://example.com
2	Worst service ever!!! 
3	None
4	Amazing experience, will buy again!!!
5	Check this out: https://test.com #awesome
6	Not good, very bad service
7	OK product, nothing special
8	I LOVE THIS PRODUCT!  http://example.com
2	Worst service ever!!! 

Cleaned & Processed Data:

post_id	text
1	I love this product!  http://example.com
2	Worst service ever!!! 
4	Amazing experience, will buy again!!!
5	Check this out: https://test.com #awesome
6	Not good, very bad service
7	OK product, nothing special
8	I LOVE THIS PRODUCT!  http://example.com

clean_text	tokens
i love this product	[love, product]
worst service ever	[worst, service]
amaz experience will buy again	[amaz, experi, buy]
check this out awesome	[check, awesome]
not good very bad service	[good, bad, service]
ok product nothing special	[ok, product, special]
i love this product	[love, product]

processed_text	word_count	sentiment_score
love product	2	5.000000e-01
worst servic	2	-1.000000e+00
amaz experi buy	3	0.000000e+00
check awesome	2	0.000000e+00
good bad servic	3	5.551115e-17
ok product special	3	4.285714e-01
love product	2	5.000000e-01

Variables

Terminal

9:46 PM Python 3

processed_text	word_count	sentiment_score
love product	2	5.000000e-01
worst servic	2	-1.000000e+00
amaz experi buy	3	0.000000e+00
check awesome	2	0.000000e+00
good bad servic	3	5.551115e-17
ok product special	3	4.285714e-01
love product	2	5.000000e-01